

Algorithm 1: Powersort($A[1..n]$)

```
1:  $X :=$  stack of runs;                                //capacity  $\lceil \lg n(n) \rceil + 1$ 
2:  $P :=$  stack of powers;                                //capacity  $\lceil \lg n(n) \rceil + 1$ 
3:  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n);$ 
4: while  $e_1 < n$  do
5:    $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n);$  //  $A[s_2..e_2]$  next run
6:    $p := \text{NodePower}(s_1, e_1, s_2, e_2, n);$  //  $P_j$  for node  $j$  between  $A[s_1..e_1]$ 
   and  $A[s_2..e_2]$ 
7:   while  $P_{\text{top}}() > p$  do
8:     //previous merge deeper in tree than current
9:      $P_{\text{pop}}();$  //  $\rightarrow$  merge and replace run  $A[s_1..e_1]$  by result
10:     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1]);$ 
11:   end
12:    $X.\text{push}(A[s_1, e_1]);$ 
13:    $P.\text{push}(p);$ 
14:    $s_1 := s_2; e_1 := e_2;$  //Now  $A[s_1..s_1]$  is the rightmost run
15: end
16: while  $\neg X.\text{empty}()$  do
17:    $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1]);$ 
18: end
```